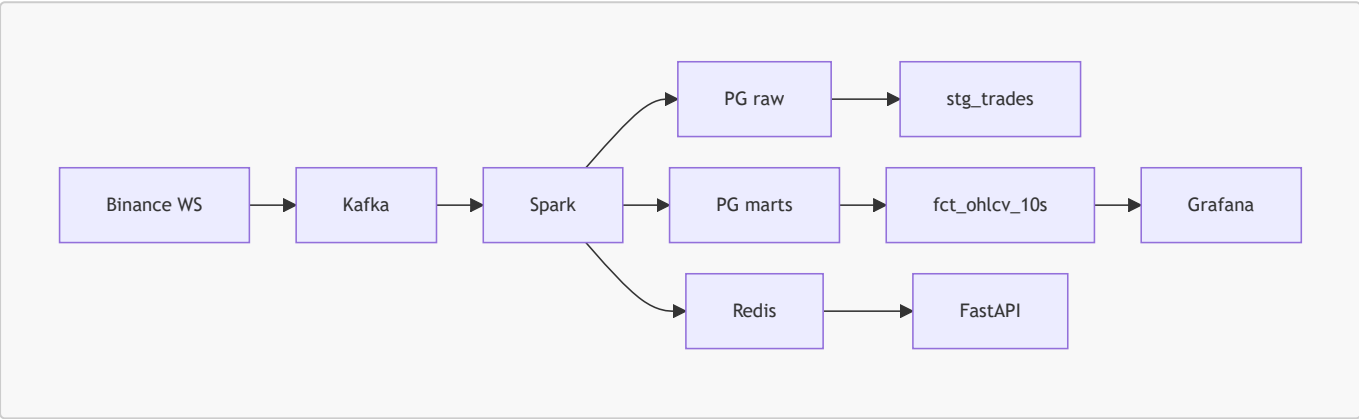


Real-Time Crypto Streaming Pipeline

A production-style, end-to-end streaming data platform that ingests live trade data from Binance, processes it with Apache Spark, stores it in PostgreSQL, and serves it through a low-latency REST API — all containerized with Docker.

Symbols tracked: BTCUSDT · ETHUSDT · SOLUSDT

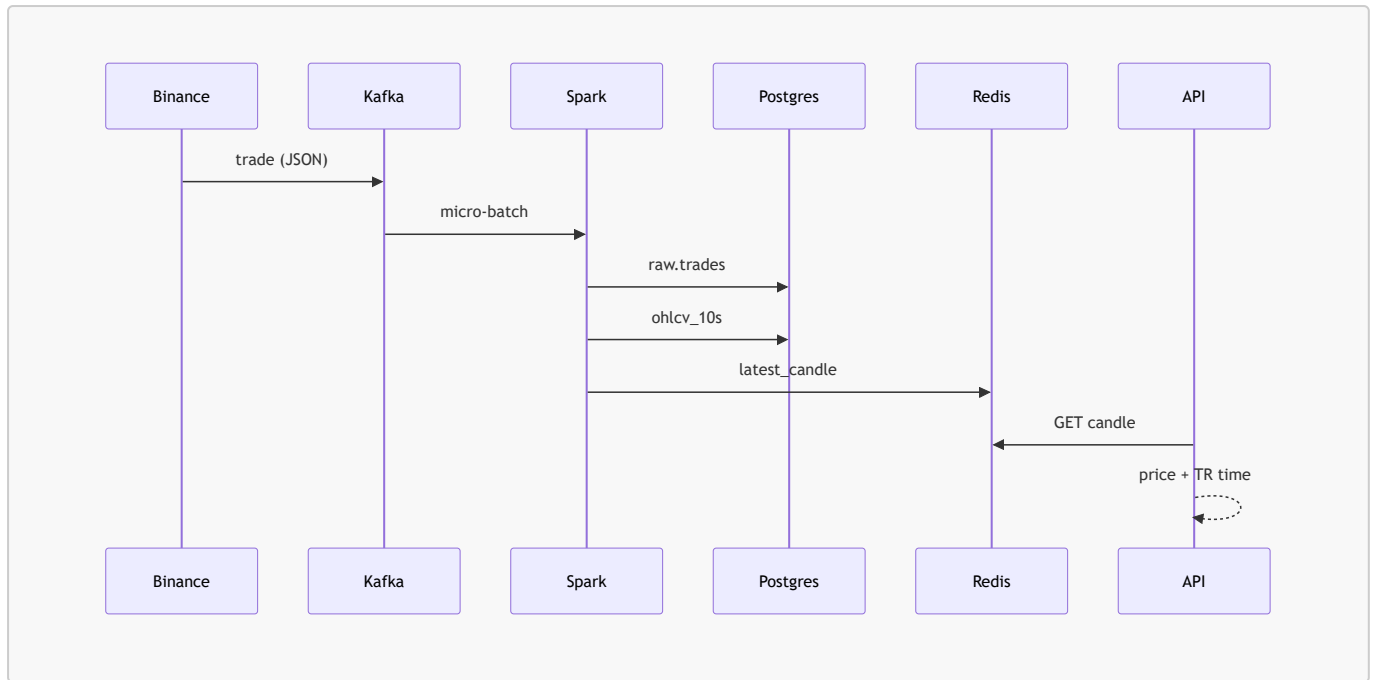
Architecture Overview



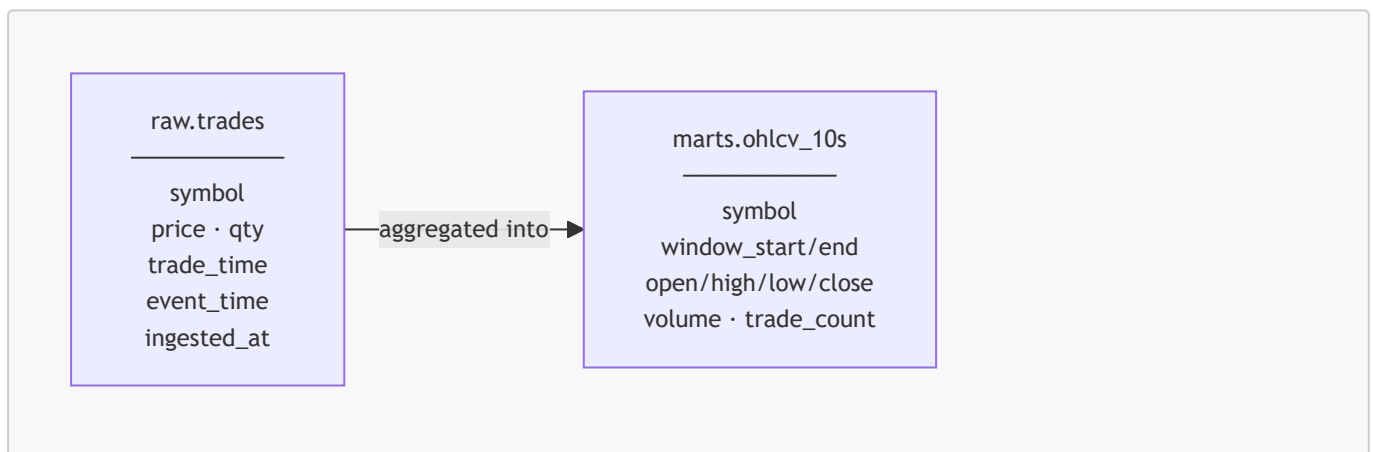
Tech Stack

Layer	Technology
Ingestion	Python · Binance WebSocket · confluent-kafka
Message Queue	Apache Kafka + Zookeeper (Confluent 7.6)
Stream Processing	Apache Spark Structured Streaming (PySpark)
Storage	PostgreSQL 16
Caching	Redis 7.2
Transformation	dbt (views on top of PostgreSQL)
API	FastAPI
Visualization	Grafana 10.4
Orchestration	Docker Compose

Data Flow



Database Schema



Project Structure

```

.
├── producer/
│   └── binance_producer.py      # WebSocket → Kafka
├── spark/
│   ├── stream_processor.py     # Kafka → Postgres + Redis
│   └── update_latest_candles.py # Fallback Redis writer
├── api/
│   └── main.py                  # FastAPI endpoints
├── dbt/
│   ├── models/staging/
│   │   └── stg_trades.sql
│   └── models/marts/
│       └── fct_ohlcv_10s.sql
└── init/
  
```

```
|   └─ init.sql           # DB schema bootstrap
|   └─ docker-compose.yml
```

Getting Started

Prerequisites

- Docker & Docker Compose
- `.env` file (see below)

Environment Variables

Create a `.env` file in the project root:

```
KAFKA_EXTERNAL_PORT=9092

POSTGRES_DB=crypto
POSTGRES_USER=postgres
POSTGRES_PASSWORD=yourpassword
POSTGRES_PORT=5432

REDIS_PORT=6379
GRAFANA_PORT=3000
API_PORT=8000

REFRESH_SECONDS=10
```

Run

```
# Start all services
docker compose up -d

# Check logs
docker compose logs -f spark-processor
docker compose logs -f producer
```

API Endpoints

Base URL: `http://localhost:8000`

Method	Endpoint	Description
GET	<code>/symbols</code>	List all tracked symbols
GET	<code>/latest-candle/{symbol}</code>	Full OHLCV candle for a symbol

Method	Endpoint	Description
GET	/latest-price/{symbol}	Close price + UTC & Istanbul time

Example:

```
curl http://localhost:8000/latest-price/BTCUSDT
```

```
{
  "symbol": "BTCUSDT",
  "price": 67432.10,
  "time_utc": "2024-05-01T12:00:00+00:00",
  "time_tr": "2024-05-01T15:00:00+03:00"
}
```

dbt Models

stg_trades (view · staging schema)

Cleaned and enriched trades from raw.trades — filters nulls, adds trade_time_second, trade_time_minute, and ingestion_delay.

fct_ohlcv_10s (view · marts schema)

Enriched OHLCV candles with derived metrics:

Column	Description
candle_type	bullish / bearish / neutral
candle_return_pct	(close - open) / open × 100
price_range	high - low
candle_body	close - open
avg_trade_size	volume / trade_count

Screenshots

Real-Time Data Dashboard

Live 10-second OHLCV candles, volume and trade count per symbol, and a live price feed — all updating in real time via Grafana.



Analytics Dashboard (dbt)

Enriched metrics powered by dbt views: candle return %, price range, avg trade size, candle type distribution, and a stat panel for BTCUSDT.



Write Latency

Measured streaming latency from OHLCV **window_end** to PostgreSQL write timestamps. First write of most 10-second candles completed within **1-2 seconds** after the window closed. Some windows show **last_update_lag** around **11 seconds**, reflecting Spark's upsert updates for recently closed aggregation windows.

	 symbol	 window_start	 window_end	 created_at	 updated_at	 first_write_lag	 last_update_lag
1	BTCUSDT	2026-04-30 01:54:20.000 +0300	2026-04-30 01:54:30.000 +0300	2026-04-30 01:54:31.480 +0300	2026-04-30 01:54:31.480 +0300	00:00:01.480718	00:00:01.480718
2	SOLUSDT	2026-04-30 01:54:20.000 +0300	2026-04-30 01:54:30.000 +0300	2026-04-30 01:54:31.475 +0300	2026-04-30 01:54:31.475 +0300	00:00:01.475361	00:00:01.475361
3	ETHUSDT	2026-04-30 01:54:20.000 +0300	2026-04-30 01:54:30.000 +0300	2026-04-30 01:54:31.475 +0300	2026-04-30 01:54:31.475 +0300	00:00:01.475361	00:00:01.475361
4	SOLUSDT	2026-04-30 01:54:10.000 +0300	2026-04-30 01:54:20.000 +0300	2026-04-30 01:54:21.391 +0300	2026-04-30 01:54:21.391 +0300	00:00:01.391384	00:00:01.391384
5	ETHUSDT	2026-04-30 01:54:10.000 +0300	2026-04-30 01:54:20.000 +0300	2026-04-30 01:54:21.391 +0300	2026-04-30 01:54:21.391 +0300	00:00:01.391384	00:00:01.391384
6	BTCUSDT	2026-04-30 01:54:10.000 +0300	2026-04-30 01:54:20.000 +0300	2026-04-30 01:54:21.388 +0300	2026-04-30 01:54:31.475 +0300	00:00:01.38878	00:00:11.475361
7	ETHUSDT	2026-04-30 01:54:00.000 +0300	2026-04-30 01:54:10.000 +0300	2026-04-30 01:54:11.622 +0300	2026-04-30 01:54:11.622 +0300	00:00:01.622754	00:00:01.622754
8	BTCUSDT	2026-04-30 01:54:00.000 +0300	2026-04-30 01:54:10.000 +0300	2026-04-30 01:54:11.622 +0300	2026-04-30 01:54:11.622 +0300	00:00:01.622754	00:00:01.622754
9	SOLUSDT	2026-04-30 01:54:00.000 +0300	2026-04-30 01:54:10.000 +0300	2026-04-30 01:54:11.575 +0300	2026-04-30 01:54:11.575 +0300	00:00:01.575502	00:00:01.575502
10	ETHUSDT	2026-04-30 01:53:50.000 +0300	2026-04-30 01:54:00.000 +0300	2026-04-30 01:54:01.954 +0300	2026-04-30 01:54:01.954 +0300	00:00:01.954687	00:00:01.954687
11	SOLUSDT	2026-04-30 01:53:50.000 +0300	2026-04-30 01:54:00.000 +0300	2026-04-30 01:54:01.954 +0300	2026-04-30 01:54:01.954 +0300	00:00:01.954687	00:00:01.954687
12	BTCUSDT	2026-04-30 01:53:50.000 +0300	2026-04-30 01:54:00.000 +0300	2026-04-30 01:54:01.954 +0300	2026-04-30 01:54:11.575 +0300	00:00:01.954687	00:00:11.575502
13	ETHUSDT	2026-04-30 01:53:40.000 +0300	2026-04-30 01:53:50.000 +0300	2026-04-30 01:53:51.421 +0300	2026-04-30 01:53:51.421 +0300	00:00:01.4217	00:00:01.4217
14	BTCUSDT	2026-04-30 01:53:40.000 +0300	2026-04-30 01:53:50.000 +0300	2026-04-30 01:53:51.421 +0300	2026-04-30 01:54:01.954 +0300	00:00:01.4217	00:00:11.954687
15	SOLUSDT	2026-04-30 01:53:40.000 +0300	2026-04-30 01:53:50.000 +0300	2026-04-30 01:53:51.413 +0300	2026-04-30 01:53:51.413 +0300	00:00:01.413979	00:00:01.413979
16	SOLUSDT	2026-04-30 01:53:30.000 +0300	2026-04-30 01:53:40.000 +0300	2026-04-30 01:53:41.313 +0300	2026-04-30 01:53:41.313 +0300	00:00:01.313195	00:00:11.413979
17	ETHUSDT	2026-04-30 01:53:30.000 +0300	2026-04-30 01:53:40.000 +0300	2026-04-30 01:53:41.313 +0300	2026-04-30 01:53:41.313 +0300	00:00:01.313195	00:00:01.313195
18	BTCUSDT	2026-04-30 01:53:30.000 +0300	2026-04-30 01:53:40.000 +0300	2026-04-30 01:53:41.308 +0300	2026-04-30 01:53:51.421 +0300	00:00:01.308537	00:00:11.4217
19	ETHUSDT	2026-04-30 01:53:20.000 +0300	2026-04-30 01:53:30.000 +0300	2026-04-30 01:53:31.535 +0300	2026-04-30 01:53:31.535 +0300	00:00:01.53552	00:00:01.53552
20	SOLUSDT	2026-04-30 01:53:20.000 +0300	2026-04-30 01:53:30.000 +0300	2026-04-30 01:53:31.535 +0300	2026-04-30 01:53:41.313 +0300	00:00:01.53552	00:00:11.313195
21	BTCUSDT	2026-04-30 01:53:20.000 +0300	2026-04-30 01:53:30.000 +0300	2026-04-30 01:53:31.521 +0300	2026-04-30 01:53:31.521 +0300	00:00:01.521812	00:00:01.521812
22	SOLUSDT	2026-04-30 01:53:10.000 +0300	2026-04-30 01:53:20.000 +0300	2026-04-30 01:53:21.220 +0300	2026-04-30 01:53:21.220 +0300	00:00:01.220089	00:00:01.220089
23	BTCUSDT	2026-04-30 01:53:10.000 +0300	2026-04-30 01:53:20.000 +0300	2026-04-30 01:53:21.220 +0300	2026-04-30 01:53:21.220 +0300	00:00:01.220089	00:00:01.220089

Screenshots were captured live while the pipeline was running. Place image files under `docs/screenshots/` in your repo.

Key Design Decisions

Watermark + update mode — Spark processes OHLCV windows with a 5-second watermark, tolerating slight late arrivals while allowing the `update` output mode for efficient upserts.

Redis as the serving layer — Spark writes the latest candle directly to Redis after each micro-batch, keeping API latency in single-digit milliseconds regardless of PostgreSQL query load.

Upsert on conflict — OHLCV records use `ON CONFLICT (symbol, window_start, window_end) DO UPDATE`, so reprocessing is idempotent.

Fallback writer — `update_latest_candles.py` can re-sync Redis from PostgreSQL independently of Spark, useful during restarts or debugging.

License

MIT